

TUGAS PRAKTIKUM 5

SISTEM OPERASI



MUHAMMAD MAULANA NUR SULAIMAN

2410651033

UNIVERITAS MUHAMMDIYAH JEMBER

FAKULTAS TEKNIK

PROGRAM STUDI TEKNIK INFORMATIKA

2025

SISTEM BOOKING HOTEL (Akhiran NIM Ganjil)

Deskripsi Masalah:

Anda diminta membuat simulasi sistem booking hotel dengan kondisi:

Hotel memiliki 5 kamar yang bisa di-booking

Ada 10 customer yang mencoba booking secara bersamaan (10 threads)

setiap customer akan:

1. Mencoba booking kamar (butuh waktu random 1-3 detik)
2. Jika berhasil, "menginap" selama 5 detik
3. Check-out dan melepaskan kamar
4. Jika gagal (kamar penuh), customer menunggu 2 detik lalu coba lagi

Customer maksimal mencoba 3 kali, jika gagal terus maka menyerah

- a. Buat file program:

```
nano sistem_booking_hotel.c
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
#include <semaphore.h>
```

```
#include <time.h>
```

```
#define JUMLAH_KAMAR 5
```

```
#define JUMLAH_CUSTOMER 10
```

```
#define MAX_ATTEMPT 3
```

```
sem_t kamar_tersedia;      // Semaphore untuk jumlah kamar
```

```
pthread_mutex_t print_mutex; // Mutex untuk output agar tidak berantakan
```

```
int kamar[JUMLAH_KAMAR] = {0}; // 0 = kosong, selain itu = ID
customer
```

```
void* customer_func(void* arg) {
    int id = *((int*)arg);
    int attempt = 0;

    while (attempt < MAX_ATTEMPT) {
        attempt++;

        pthread_mutex_lock(&print_mutex);
        printf("[🕒 %02d:%02d:%02d] 🧑 Customer-%d mencoba booking
(percobaan %d)...\n",
            localtime(&(time_t){time(NULL)})->tm_hour,
            localtime(&(time_t){time(NULL)})->tm_min,
            localtime(&(time_t){time(NULL)})->tm_sec,
            id, attempt);
        pthread_mutex_unlock(&print_mutex);

        sleep(rand() % 3 + 1); // waktu random 1–3 detik

        // Coba ambil kamar
        if (sem_trywait(&kamar_tersedia) == 0) {
            // Dapat kamar
            int kamar_id = -1;
            for (int i = 0; i < JUMLAH_KAMAR; i++) {
                if (kamar[i] == 0) {
                    kamar[i] = id;
                    kamar_id = i + 1;
                    break;
                }
            }
        }
    }
}
```

```

    }
}

int value;
sem_getvalue(&kamar_tersedia, &value);

pthread_mutex_lock(&print_mutex);
printf("[🕒 %02d:%02d:%02d] ✅ Customer-%d BERHASIL
booking kamar #%d! (%d kamar tersisa)\n",
    localtime(&(time_t){time(NULL)})->tm_hour,
    localtime(&(time_t){time(NULL)})->tm_min,
    localtime(&(time_t){time(NULL)})->tm_sec,
    id, kamar_id, value);
pthread_mutex_unlock(&print_mutex);

// Menginap selama 5 detik
sleep(5);

// Check-out
kamar[kamar_id - 1] = 0;
sem_post(&kamar_tersedia);
sem_getvalue(&kamar_tersedia, &value);

pthread_mutex_lock(&print_mutex);
printf("[🕒 %02d:%02d:%02d] 📅 Customer-%d check-out dari
kamar #%d (%d kamar tersisa)\n",
    localtime(&(time_t){time(NULL)})->tm_hour,
    localtime(&(time_t){time(NULL)})->tm_min,
    localtime(&(time_t){time(NULL)})->tm_sec,
    id, kamar_id, value);
pthread_mutex_unlock(&print_mutex);

```

```

        pthread_exit(NULL);
    } else {
        // Gagal booking
        pthread_mutex_lock(&print_mutex);

        printf("[🕒 %02d:%02d:%02d] ❌ Customer-%d gagal booking
(kamar penuh), menunggu 2 detik...\n",
            localtime(&(time_t){time(NULL)})->tm_hour,
            localtime(&(time_t){time(NULL)})->tm_min,
            localtime(&(time_t){time(NULL)})->tm_sec,
            id);
        pthread_mutex_unlock(&print_mutex);
        sleep(2);
    }
}

pthread_mutex_lock(&print_mutex);
printf("[🕒 %02d:%02d:%02d] 😞 Customer-%d menyerah setelah %d
kali mencoba.\n",
    localtime(&(time_t){time(NULL)})->tm_hour,
    localtime(&(time_t){time(NULL)})->tm_min,
    localtime(&(time_t){time(NULL)})->tm_sec,
    id, MAX_ATTEMPT);
pthread_mutex_unlock(&print_mutex);

pthread_exit(NULL);
}

int main() {
    srand(time(NULL));

```

```

printf("=== 🏨 SISTEM BOOKING HOTEL ===\n");
printf("Total Kamar: %d\n\n", JUMLAH_KAMAR);

pthread_t threads[JUMLAH_CUSTOMER];
int id[JUMLAH_CUSTOMER];

sem_init(&kamar_tersedia, 0, JUMLAH_KAMAR);
pthread_mutex_init(&print_mutex, NULL);

for (int i = 0; i < JUMLAH_CUSTOMER; i++) {
    id[i] = i + 1;
    pthread_create(&threads[i], NULL, customer_func, &id[i]);
    usleep(100000); // sedikit jeda agar output rapi
}

for (int i = 0; i < JUMLAH_CUSTOMER; i++) {
    pthread_join(threads[i], NULL);
}

sem_destroy(&kamar_tersedia);
pthread_mutex_destroy(&print_mutex);

printf("\n=== Semua customer selesai. Hotel kosong kembali. ===\n");
return 0;
}

```

- Semaphore (sem_t kamar_tersedia): menjaga jumlah kamar maksimum = 5.
 - *sem_wait()* → customer ambil kamar.
 - *sem_post()* → customer check-out, kamar bertambah lagi.

- Mutex (*print_mutex*): agar output tidak tumpang tindih antar thread.
- *sem_trywait()*: mencoba booking tanpa blocking; kalau kamar penuh → gagal.
- 3 kali percobaan: setelah 3 kali gagal, customer menyerah.
- Sleep acak: mensimulasikan waktu proses booking & menginap.

Dalam program ini, potensi *race condition* muncul karena beberapa *thread* (customer) berjalan bersamaan dan dapat:

- Mencetak output ke layar secara serentak.
- Mengakses dan mengubah status kamar (array kamar[]) pada waktu yang sama.

Untuk **menghindari race condition**, digunakan dua mekanisme sinkronisasi utama:

1. Semaphore (*sem_t* kamar_tersedia)
 - Mengontrol jumlah kamar yang bisa digunakan bersamaan.
 - Setiap kali customer berhasil booking, semaphore dikurangi (*sem_wait()* atau *sem_trywait()*), dan saat check-out semaphore dikembalikan (*sem_post()*).
 - Ini mencegah dua customer memesan kamar yang sama di waktu bersamaan.
2. Mutex (*pthread_mutex_t* *print_mutex*)
 - Digunakan untuk melindungi bagian *print/output* agar hanya satu *thread* yang menulis ke layar pada satu waktu.
 - Tanpa mutex, teks dari beberapa *thread* bisa tercampur (berantakan dan sulit dibaca).

Dengan kombinasi semaphore dan mutex, program menjadi aman dari *race condition* baik pada data kamar maupun output di terminal.

Dan jika tidak menggunakan mekanisme sinkronisasi, maka akan terjadi beberapa masalah:

1. Data kamar bisa tumpang tindih (inkonsisten)
 - Dua atau lebih customer bisa “berhasil” memesan kamar yang sama karena tidak ada batasan akses bersamaan ke variabel kamar[].

- Akibatnya, jumlah kamar tersisa tidak sesuai kenyataan.
2. Output di layar menjadi kacau
 - Karena semua *thread* mencetak teks tanpa mutex, hasil print bisa bercampur antara beberapa baris, membuat log sulit dibaca.
 3. Logika program salah atau crash
 - Nilai semaphore atau data kamar bisa berubah tidak menentu akibat beberapa *thread* menulis bersamaan, yang dapat menyebabkan hasil yang tidak dapat diprediksi (nondeterministic behavior).

Hasil:

```

alangananteng@LAPTOP-CV1QG677:~$ nano sistem_booking_hotel.c
alangananteng@LAPTOP-CV1QG677:~$ gcc sistem_booking_hotel.c -o sistem_booking_hotel -pthread
alangananteng@LAPTOP-CV1QG677:~$ ./sistem_booking_hotel
=== 🏨 SISTEM BOOKING HOTEL ===
Total Kamar: 5

[⌚ 19:13:57] 🧑 Customer-1 mencoba booking (percobaan 1)...
[⌚ 19:13:57] 🧑 Customer-2 mencoba booking (percobaan 1)...
[⌚ 19:13:58] 🧑 Customer-3 mencoba booking (percobaan 1)...
[⌚ 19:13:58] 🧑 Customer-4 mencoba booking (percobaan 1)...
[⌚ 19:13:58] 🧑 Customer-5 mencoba booking (percobaan 1)...
[⌚ 19:13:58] 🧑 Customer-6 mencoba booking (percobaan 1)...
[⌚ 19:13:58] 🧑 Customer-7 mencoba booking (percobaan 1)...
[⌚ 19:13:58] 🧑 Customer-8 mencoba booking (percobaan 1)...
[⌚ 19:13:58] 🧑 Customer-9 mencoba booking (percobaan 1)...
[⌚ 19:13:58] 🧑 Customer-10 mencoba booking (percobaan 1)...
[⌚ 19:13:59] ✅ Customer-3 BERHASIL booking kamar #1! (4 kamar tersisa)
[⌚ 19:13:59] ✅ Customer-4 BERHASIL booking kamar #2! (3 kamar tersisa)
[⌚ 19:13:59] ✅ Customer-5 BERHASIL booking kamar #3! (2 kamar tersisa)
[⌚ 19:13:59] ✅ Customer-8 BERHASIL booking kamar #4! (1 kamar tersisa)
[⌚ 19:13:59] ✅ Customer-10 BERHASIL booking kamar #5! (0 kamar tersisa)
[⌚ 19:13:59] ❌ Customer-2 gagal booking (kamar penuh), menunggu 2 detik...
[⌚ 19:14:00] ❌ Customer-7 gagal booking (kamar penuh), menunggu 2 detik...
[⌚ 19:14:00] ❌ Customer-1 gagal booking (kamar penuh), menunggu 2 detik...
[⌚ 19:14:01] ❌ Customer-6 gagal booking (kamar penuh), menunggu 2 detik...
[⌚ 19:14:01] ❌ Customer-9 gagal booking (kamar penuh), menunggu 2 detik...
[⌚ 19:14:01] 🧑 Customer-2 mencoba booking (percobaan 2)...
[⌚ 19:14:02] 🧑 Customer-7 mencoba booking (percobaan 2)...
[⌚ 19:14:02] 🧑 Customer-1 mencoba booking (percobaan 2)...
[⌚ 19:14:02] ❌ Customer-2 gagal booking (kamar penuh), menunggu 2 detik...
[⌚ 19:14:03] 🧑 Customer-6 mencoba booking (percobaan 2)...
[⌚ 19:14:03] ❌ Customer-7 gagal booking (kamar penuh), menunggu 2 detik...
[⌚ 19:14:03] 🧑 Customer-9 mencoba booking (percobaan 2)...
[⌚ 19:14:04] 📄 Customer-3 check-out dari kamar #1 (1 kamar tersisa)
[⌚ 19:14:04] 📄 Customer-4 check-out dari kamar #2 (2 kamar tersisa)
[⌚ 19:14:04] 📄 Customer-5 check-out dari kamar #3 (3 kamar tersisa)
[⌚ 19:14:04] 📄 Customer-8 check-out dari kamar #4 (4 kamar tersisa)
[⌚ 19:14:04] 📄 Customer-10 check-out dari kamar #5 (5 kamar tersisa)
[⌚ 19:14:04] 🧑 Customer-2 mencoba booking (percobaan 3)...
[⌚ 19:14:05] ✅ Customer-6 BERHASIL booking kamar #1! (4 kamar tersisa)
[⌚ 19:14:05] 🧑 Customer-7 mencoba booking (percobaan 3)...
[⌚ 19:14:05] ✅ Customer-1 BERHASIL booking kamar #2! (3 kamar tersisa)
[⌚ 19:14:06] ✅ Customer-7 BERHASIL booking kamar #3! (2 kamar tersisa)
[⌚ 19:14:06] ✅ Customer-9 BERHASIL booking kamar #4! (1 kamar tersisa)
[⌚ 19:14:07] ✅ Customer-2 BERHASIL booking kamar #5! (0 kamar tersisa)
[⌚ 19:14:10] 📄 Customer-6 check-out dari kamar #1 (1 kamar tersisa)
[⌚ 19:14:10] 📄 Customer-1 check-out dari kamar #2 (2 kamar tersisa)

[⌚ 19:14:11] 📄 Customer-7 check-out dari kamar #3 (3 kamar tersisa)
[⌚ 19:14:11] 📄 Customer-9 check-out dari kamar #4 (4 kamar tersisa)
[⌚ 19:14:12] 📄 Customer-2 check-out dari kamar #5 (5 kamar tersisa)

=== Semua customer selesai. Hotel kosong kembali. ===

```


